

reserve-gate

Recording-ready five-minute video script

Exact page, exact click, exact words - verified against the deployed site on 4 September 2026.

HOW TO READ THIS

Blue ON SCREEN instructions are actions; do not say them. Red SAY text is your voice. CURSOR CUE tells you where to point while speaking. A slash / means a short pause.

Target

Aim for 4:35 to 4:55 after cutting loading time. Speak clearly, not quickly. The script is intentionally shorter than five minutes so cursor movement and small pauses fit.

Critical wording

DO NOT SAY 'HOLD -> CAPTURE -> SPENT'

On this site, HOLD means waiting for human approval. The payment lifecycle uses a different idea: reserve the amount, bind it to one order, then commit it after capture. In the video, say 'reserve -> capture -> commit.'

DO NOT CALL THE GUIDED DEMO A LIVE AI RUN

The button is named 'Run the AI shopping trip,' but this section submits a fixed catalogue basket through the real policy and ledger. Describe it as a deterministic guided walkthrough. The real-model results appear later as committed evidence, not as a live model call on this page.

Prepare the browser before recording

Use 1920 x 1080, browser zoom 90% to 100%, and keep the cursor visible. Close personal tabs, notifications, email, terminals, .env files, and password managers.

Open these four tabs in this order

1. Home: <https://reserve-gate.onrender.com/>
2. Guided Demo: <https://reserve-gate.onrender.com/demo>
3. Follow the Money: <https://reserve-gate.onrender.com/trace>
4. The Proof: <https://reserve-gate.onrender.com/evidence>

Prepare tab 3

Wait until the purchase list loads. Confirm a live test purchase - ₹100.00 - paid and charged is selected. Scroll until What was written down and its four events are visible.

Prepare tab 4

For the architecture shot, scroll to The full reports, open How the whole thing is put together, and stop with the diagram titled Shape - every request passes through one gate visible. After that shot, press Home to return to the top for the evidence numbers later.

RECORDING RULE

Do not wait through loading in the final video. Pause speaking, let the action finish, and remove the dead time during editing. Do not cut between a click and the result so tightly that the result looks fake.

If Razorpay checkout fails

Do not keep the failed attempt in the final video. Use the recorded ₹100 purchase on the Trace page and say: 'The live checkout did not complete in this take, so I am showing the committed recording of the same test-mode flow.'

The complete recording script

0:00-0:25 | Introduction and problem

ON SCREEN	TAB 1 - HOME. Start at the top. Keep the headline AI spends. You set the ceiling. fully visible. Do not scroll until the final sentence.
SAY	Hello, I am Sumeet. / This is reserve-gate, my project for Track 01, AI Growth and Agentic Commerce. An AI shopping agent may receive payment tools, but a spending limit written only inside its prompt is not a real financial control. If the model makes a mistake or its prompt is changed, that instruction can fail.
CURSOR CUE	Point once at A limit in a prompt is a request. Then click How it works in the top navigation.

0:25-0:50 | The solution

ON SCREEN	HOME - HOW RESERVE-GATE WORKS. Keep the three rows You set the limits, The AI proposes, and The gate decides visible.
SAY	reserve-gate sits between the AI buyer and Razorpay. The AI may propose a purchase, but it cannot choose its own authority. Fixed code checks every money request against the budget and policy before anything can reach Razorpay. The answer is always allow, wait for human approval, or block.
CURSOR CUE	Move down the three rows in order. Do not wave the cursor.

0:50-1:18 | Architecture

ON SCREEN	SWITCH TO TAB 4 - THE PROOF. Show the already-open architecture diagram. Keep Buyer agent, reserve-gate, and the BLOCK / HOLD / ALLOW outputs visible. Do not scroll into the long architecture text.
SAY	This architecture supports either a scripted buyer or a model-driven buyer. Every request enters reserve-gate through the same boundary. Deterministic policy checks budget, expiry, approval, replay, and reconciliation rules. SQLite changes the balance inside a transaction, so two requests cannot spend the same money. Only an allowed payment request may continue to Razorpay test mode, and every outcome is appended to the audit chain.
CURSOR CUE	Trace left to right: Buyer agent, reserve-gate, then the three outcomes.

1:18-1:42 | Set the limits

ON SCREEN	SWITCH TO TAB 2 - GUIDED DEMO. Click Limits in the checkpoint navigation. Keep all three fields visible: Total budget ₹10,000; Maximum single purchase ₹5,000; Ask me above ₹2,000. Click Apply my limits once and wait for Limits applied. Your route is ready.
SAY	I will give this AI a total budget of ten thousand rupees. One purchase cannot exceed five thousand, and anything above two thousand must wait for my approval. Applying these limits creates a fresh spending block for this browser.
CURSOR CUE	Point to each value as you name it, then click the button.

1:42-2:12 | Run the shopping trip

ON SCREEN	Click Plan. Keep the three preview items and Run the AI shopping trip visible. Click it once. Stop speaking while it runs; cut that wait. Continue only after Six decisions complete.
SAY	This guided walkthrough now submits six fixed desk-setup requests. No model chooses the items in this section; keeping the basket deterministic gives every judge the same result. All six requests still pass through the real policy and ledger. Five receive ALLOW, shown here as Purchased in the sandbox. The second monitor arm is refused because only one thousand and fifty rupees remain.
CURSOR CUE	After the result arrives, click Decisions and frame 5 sales - ₹8,950.00 plus the final blocked row.

2:12-2:38 | Show one safe failure

ON SCREEN	At the final row, keep Blocked, R1, a second monitor arm, and ₹2,000.00 visible. Click its Why? control so the ₹1,050 reason appears.
SAY	This is one failure handled safely. The request was two thousand rupees, but only one thousand and fifty remained. Rule R1 blocked it before payment, no money moved, and the exact reason came back to the AI. Product names and notes cannot change this decision because free text is not used by the spending rules.
CURSOR CUE	Point to R1, then the requested amount, then the expanded ₹1,050 reason.

2:38-3:05 | Razorpay test payment

ON SCREEN	Click Payment. Keep TEST MODE - NO REAL MONEY, ₹100, and the payment button visible. Click Pay ₹100 with a test card. In Razorpay Checkout, use card 4100 2800 0000 1007, any future expiry, and any three-digit CVV. Cut loading time; keep the successful result.
SAY	This is Razorpay's real Checkout interface running in test mode, so no real money moves. This fixed verification purchase is not chosen by an AI. reserve-gate first reserves one hundred rupees, creates the order, and records the payment only after Razorpay confirms capture. This proves the gated payment route into Razorpay.
CURSOR CUE	Do not read the test card number aloud. Let the Razorpay branding and success state remain visible briefly.

3:05-3:53 | The hardest engineering problem

ON SCREEN	SWITCH TO TAB 3 - FOLLOW THE MONEY. The paid ₹100 purchase should be selected. Keep Policy verdict - ALLOW, Order ID bound, and Debit committed visible. After saying 'never paid,' scroll up, click monitor arm - ₹2,000.00 - never paid, then return to its shorter event list.
SAY	The hardest engineering problem was deciding exactly when money becomes spent. At order creation, reserve-gate does not immediately mark the amount as spent. It reserves the amount and binds that reservation to one Razorpay order, so another request cannot reuse it. Only verified capture changes that same reservation into spent; on this page, that is Debit committed. Now compare this never-paid order: it has no Debit committed event, so it was never counted as spent. An expired unpaid reservation is later released.
CURSOR CUE	Paid trace: point to Order ID bound, then Debit committed. Unpaid trace: move to the end and visibly show that Debit committed is absent.

3:53-4:25 | Measured evidence

ON SCREEN	SWITCH TO TAB 4 - THE PROOF. Press Home first if needed. Keep The claim, in four numbers visible: 150, 0, 0/58, and 6/6.
SAY	I tested the real policy and ledger with one hundred and fifty difficult purchases. Zero unsafe purchases got through, and none of the fifty-eight honest purchases was wrongly refused. In a separate committed evaluation, the same over-limit request was sent through six different models. These are recorded results, not a live call on this page. reserve-gate refused all six through the same tool boundary.
CURSOR CUE	Move across the four numbers slowly from left to right.

4:25-4:42 | Mutation proof

ON SCREEN	SWITCH TO TAB 1 - HOME. Click Proof in the navigation. Frame All sixteen guards deleted beside All sixteen guards on, including 37, ₹68,502, 0, and ₹0.
SAY	I then deleted all sixteen guards and replayed the same cases. Thirty-seven unsafe money calls worth sixty-eight thousand five hundred and two rupees passed. With every guard restored, both numbers returned to zero.
CURSOR CUE	Point first to the deleted-guard figures, then to the all-guards-on zeroes.

4:42-4:58 | Honest limitation and ending

ON SCREEN	On HOME, press Home to return to the hero. End with AI spends. You set the ceiling. visible. Optional: cut to your face for the final sentence.
SAY	This project uses Razorpay Orders and Payments in test mode. It models Reserve Pay behaviour because Reserve Pay itself is not exposed in the test API. reserve-gate gives an AI enough authority to be useful without giving it unlimited authority over money. / Thank you.
CURSOR CUE	Stop moving the cursor for the final two sentences.

One-page recording checklist

Before pressing Record

□ Server is warm. □ Four tabs are open in order. □ Demo shows default limits. □ Trace paid purchase has loaded. □ Evidence architecture disclosure is prepared. □ Notifications are off. □ No secrets or personal tabs are visible.

During the take

□ Speak only after each page settles. □ Point at one fact at a time. □ Click each action once. □ Pause during network waits. □ Call the Guided Demo deterministic, not a live AI run. □ Call the six-model table committed evidence, not a live call. □ Never say that the AI decides whether money moves. □ Never call the approval verdict HOLD a payment reservation.

Before exporting

□ Remove loading gaps and mistakes. □ Keep clicks connected to their results. □ Check that ₹1,050, R1, Order ID bound, Debit committed, 150, 0/58, 37, and ₹68,502 are readable. □ Export MP4 at 1080p and 30 FPS. □ Listen once with headphones.

THE SINGLE MOST IMPORTANT SHOT

On the Trace page, show Order ID bound followed by Debit committed, then show a never paid purchase with no Debit committed event. That is the clearest visual proof of the hardest engineering problem.

Evidence notes

[measured 2026-09-01: harness/gate_off.py and the committed evaluation reports] The deployed site reports 150 cases, zero unsafe passes, 0/58 honest purchases refused, and 37 unsafe calls worth ₹68,502 when all sixteen guards are deleted.

[vendor-doc: <https://razorpay.com/buildathon/>] Buildathon framing and Track 01 naming. The project limitation about Reserve Pay test-API availability is stated explicitly on the deployed architecture page and in the repository documentation.